

Deep Generative Modeling

March 2026

<https://linklab.github.io>

What is Deep Generative Modeling?

What is Deep Generative Modeling?

Deep generative models (DGMs) are neural networks that learn a probability distribution over high-dimensional data (e.g., images, text, audio), so they can generate new examples that resemble the dataset.



Data Distribution

p_{data}

True underlying distribution of the data



Model Learning

p_{ϕ}

Neural network that approximates the data distribution



Generation

$x \sim p_{\phi}$

Sampling to create new examples

🎯 Goal

- Learn a tractable probability distribution p_{ϕ} from a finite dataset
- Minimize the divergence between p_{ϕ} and p_{data}
- After training, generate new samples by drawing $x \sim p_{\phi}$

✅ Model Evaluation

- Generated samples should resemble real data
- Summary statistics should match between generated and real data
- Task-specific metrics evaluate effectiveness
- Perceptual metrics assess quality

Two Goals of Generative Modeling

Deep generative models aim to achieve two fundamental objectives:



1. Realistic Generation

Generate novel, realistic samples indistinguishable from real data

- Creates samples that match the statistical properties of real data
- Uses evaluation metrics to assess quality and realism
- Focuses on capturing the entire data distribution
- Applications include image generation, text synthesis, and audio synthesis



2. Controllable Generation

Enable fine-grained and interpretable control over the generative process

- Provides mechanisms to manipulate specific aspects of generation
- Allows interpolation between different data points
- Enables conditional generation given specific attributes
- Applications include image editing, style transfer, and controlled synthesis

These goals drive the development of various generative modeling approaches

Mathematical Setup and Goals

√* Mathematical Setup

Data Assumption

- i.i.d. samples from $p_{data}(\mathbf{x})$
- Observations from unknown distribution
- Goal: Create model to approximate $p_{data}(\mathbf{x})$

Model Parameterization

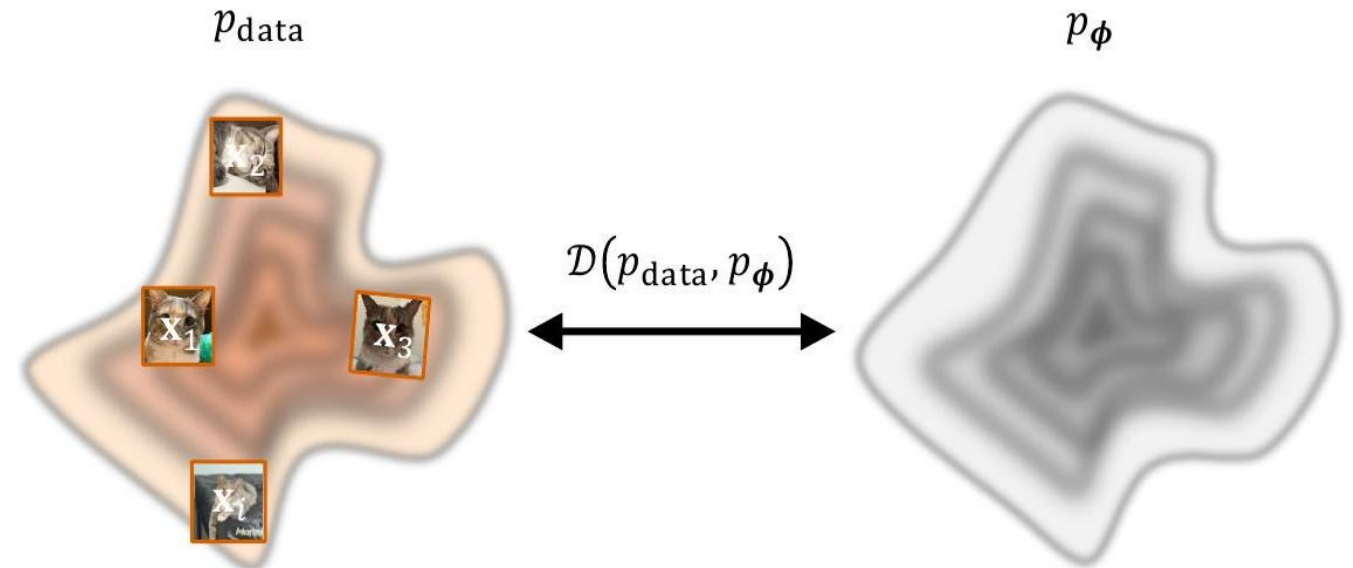
- DGM uses neural network to parameterize $p_{\phi}(\mathbf{x})$
- This model $p_{\phi}(\mathbf{x})$ is commonly referred to as a **generative model**

© Learning Objective

$$\phi^* \in \arg \min_{\phi} \mathcal{D}(p_{data}, p_{\phi})$$

When p_{ϕ} approximates p_{data} , it can:

- Generate new samples via $\mathbf{x} \sim p_{\phi}$
- Evaluate probability values $p_{\phi}(\mathbf{x})$



Choice of $\mathcal{D}(p_{data}, p_{\phi})$

Forward KL Divergence

$$\begin{aligned}
 \mathcal{D}_{KL}(p_{data} \| p_{\phi}) &:= \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{p_{\phi}(\mathbf{x})} d\mathbf{x} \\
 &= \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log p_{data}(\mathbf{x}) - \log p_{\phi}(\mathbf{x})] \\
 &= -\mathbb{E}_{\mathbf{x} \sim p_{data}} [\log p_{\phi}(\mathbf{x})] + \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log p_{data}(\mathbf{x})] \\
 &= -\mathbb{E}_{\mathbf{x} \sim p_{data}} [\log p_{\phi}(\mathbf{x})] + \underbrace{H(p_{data})}_{\substack{\uparrow \\ \text{Entropy of the data distribution} \\ \rightarrow \text{It is constant with respect to } p_{\phi}}}
 \end{aligned}$$

Minimizing KL $\Leftrightarrow$ MLE (Maximizing Log Likelihood)

$$\min_{\phi} \mathcal{D}_{KL}(p_{data} \| p_{\phi}) \Leftrightarrow \max_{\phi} \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log p_{\phi}(\mathbf{x})]$$

- Use i.i.d. samples $\{\mathbf{x}^{(i)}\}_{i=1}^N \sim p_{data}$
- Empirical MLE objective (optimized via SGD)

$$\hat{\mathcal{L}}_{MLE}(\phi) := -\frac{1}{N} \sum_{i=1}^N \log p_{\phi}(\mathbf{x}^{(i)})$$

Choice of $\mathcal{D}(p_{data}, p_{\phi})$

Forward KL Divergence

$$\mathcal{D}_{KL}(p_{data} \| p_{\phi}) := \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{p_{\phi}(\mathbf{x})} d\mathbf{x}$$

- Minimizing $\mathcal{D}_{KL}(p_{data} \| p_{\phi}) \rightarrow$ **"Mode covering"** 현상 발생
 Why? if there exist a data $\mathbf{x}$ with $p_{data}(\mathbf{x}) > 0$ but $p_{\phi}(\mathbf{x}) = 0$, then $\mathcal{D}_{KL}(p_{data} \| p_{\phi}) = +\infty$ since the integrand contains $\log \frac{p_{data}(\mathbf{x})}{p_{\phi}(\mathbf{x})} = +\infty$
- Minimizing **forward KL** forces the model to assign probability where the data has support, so that the model tries to cover all data modes



[Mode Covering]

$\mathcal{D}_{KL}(p_{data} \| p_{\phi})$ 을 최소화하려면 어쩔 수 없이, 모델 $p_{\phi}(\mathbf{x})$ 이 "데이터 $\mathbf{x}$ 가 있는 모든 곳"에 반드시 확률을 주도록 강제됨

$\rightarrow$ 하나의 mode 라도 빼먹으면 최소화 관점에서 엄청 큰 손해

Choice of $\mathcal{D}(p_{data}, p_{\phi})$

Forward KL Divergence vs. Reverse KL Divergence

$$\begin{aligned}\mathcal{D}_{KL}(p_{data} \| p_{\phi}) &:= \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{p_{\phi}(\mathbf{x})} d\mathbf{x} \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log p_{data}(\mathbf{x}) - \log p_{\phi}(\mathbf{x})]\end{aligned}$$

- Minimizing $\mathcal{D}_{KL}(p_{data} \| p_{\phi}) \rightarrow$ **"Mode covering"** 현상 발생
Why? if there exist a data $\mathbf{x}$ with $p_{data}(\mathbf{x}) > 0$ but $p_{\phi}(\mathbf{x}) = 0$, then $\mathcal{D}_{KL}(p_{data} \| p_{\phi}) = +\infty$ since the integrand contains $\log \frac{p_{data}(\mathbf{x})}{p_{\phi}(\mathbf{x})} = +\infty$
- Minimizing **forward KL** forces the model to assign probability where the data has support, so that the model tries to cover all data modes

- Minimizing $\mathcal{D}_{KL}(p_{data} \| p_{\phi}) \rightarrow$ **"Mode covering"** 현상 발생
Why? if there exist a data $\mathbf{x}$ with $p_{data}(\mathbf{x}) > 0$ but $p_{\phi}(\mathbf{x}) = 0$, then $\mathcal{D}_{KL}(p_{data} \| p_{\phi}) = +\infty$ since the integrand contains $\log \frac{p_{data}(\mathbf{x})}{p_{\phi}(\mathbf{x})} = +\infty$
- Minimizing **forward KL** forces the model to assign probability where the data has support, so that the model tries to cover all data modes

Asymmetric: $\mathcal{D}_{KL}(p_{data} \| p_{\phi}) \neq \mathcal{D}_{KL}(p_{\phi} \| p_{data})$

$$\begin{aligned}\mathcal{D}_{KL}(p_{\phi} \| p_{data}) &:= \int p_{\phi}(\mathbf{x}) \log \frac{p_{\phi}(\mathbf{x})}{p_{data}(\mathbf{x})} d\mathbf{x} \\ &= \mathbb{E}_{\mathbf{x} \sim p_{\phi}} [\log p_{\phi}(\mathbf{x}) - \log p_{data}(\mathbf{x})]\end{aligned}$$

- Minimizing $\mathcal{D}_{KL}(p_{\phi} \| p_{data}) \rightarrow$ **"Mode seeking (or Mode collapse)"** 현상 발생
Why? if there exist a data $\mathbf{x}$ with $p_{\phi}(\mathbf{x}) > 0$ but $p_{data}(\mathbf{x}) = 0$, then $\mathcal{D}_{KL}(p_{\phi} \| p_{data}) = +\infty$ since the integrand contains $\log \frac{p_{\phi}(\mathbf{x})}{p_{data}(\mathbf{x})} = +\infty$
- Minimizing **backward KL** forces the model to assign "NO" probability where the data has NOT support, so that the model tries to ignore small data modes



Choice of $\mathcal{D}(p_{data}, p_{\phi})$

Score Functions

$$\nabla_{\mathbf{x}} \log p(\mathbf{x})$$

지금 위치 $\mathbf{x}$ 에서, 확률이 더 커지는 방향은 어디인가?

- The score function is defined as the gradient of the log-density
- Score functions point toward regions of higher probability and are central to many generative models

현재 샘플을 어디로 조금 움직이면 더 그럴듯한 데이터가 되는지 알려주는 방향 벡터

Fisher Divergence

- For two distributions p and q , the Fisher divergence is defined as:

$$\mathcal{D}_F(p||q) := \mathbb{E}_{\mathbf{x} \sim p} [\|\nabla_{\mathbf{x}} \log p(\mathbf{x}) - \nabla_{\mathbf{x}} \log q(\mathbf{x})\|_2^2]$$

- It measures the discrepancy between score functions
- $\mathcal{D}_F(p||q) = 0$ if and only if $p = q$ almost everywhere

두 분포의 “확률이 증가하는 방향(score)” 을 비교 → 그 방향이 모든 곳에서 완전히 같을 때 0 → 두 분포가 동일

Invariance Property

Key Insight: Fisher divergence is invariant to normalization constants

- Scores depend only on gradients of log-densities
- Normalizing constants cancel out in the gradient
- This invariance is crucial for practical applications

$$p(\mathbf{x}) = \frac{\tilde{p}(\mathbf{x})}{Z} \quad \Rightarrow \quad \nabla_{\mathbf{x}} \log p(\mathbf{x}) = \nabla_{\mathbf{x}} \log \tilde{p}(\mathbf{x}) - \nabla_{\mathbf{x}} \log Z$$

$$\tilde{p}(\mathbf{x}): \text{unnormalized density} \quad \Rightarrow \quad \nabla_{\mathbf{x}} \log p(\mathbf{x}) = \nabla_{\mathbf{x}} \log \tilde{p}(\mathbf{x})$$

Connection to Diffusion Modeling

Importance: Fisher divergence forms the basis of **score matching**

- **Score matching** is a method that learns the gradient of the log-density for generation

Diffusion 모델은 확률밀도가 아니라 score (확률이 증가하는 방향)만 배운다.

Choice of $\mathcal{D}(p_{data}, p_{\phi})$

f-Divergence Family

✖ A broad family of divergences defined as: $\mathcal{D}_f(p\|q) = \int q(\mathbf{x}) f\left(\frac{p(\mathbf{x})}{q(\mathbf{x})}\right) d\mathbf{x}, f(1) = 0$
 where $f: \mathbb{R}_+ \rightarrow \mathbb{R}$ is a convex function with $f(1) = 0$

> KL Divergence

$$f(u) = u \log u$$

$$\mathcal{D}_{KL}(p\|q) := \int p \log \frac{p}{q} d\mathbf{x}$$

✎ Asymmetric measure that encourages mode covering

⚖ Jensen-Shannon

$$f(u) = \frac{1}{2} \left[u \log u - (u + 1) \log \frac{1+u}{2} \right]$$

$$\mathcal{D}_{JS}(p\|q) = \frac{1}{2} \mathcal{D}_{KL} \left(p \parallel \frac{1}{2}(p + q) \right) + \frac{1}{2} \mathcal{D}_{KL} \left(q \parallel \frac{1}{2}(p + q) \right)$$

✎ Smooth and symmetric measure that balances both distributions

↔ Total Variation

$$f(u) = \frac{1}{2} [u - 1]$$

$$\mathcal{D}_{TV}(p, q) = \frac{1}{2} \int_{\mathbb{R}^D} |p - q| d\mathbf{x} = \sup_{A \subset \mathbb{R}^D} |p(A) - q(A)|$$

✎ Captures largest probability difference between two distributions

💡 **Key Insight:** Each divergence embodies a different notion of closeness between distributions, inducing distinct learning behavior in generative models.

Challenges in Distribution Modeling

⊕ Non-Negativity

Neural networks can produce negative outputs, violating the requirement that probability densities must be non-negative:

$$p_{\phi}(\mathbf{x}) \geq 0$$

⚖ Normalization

Neural networks can produce a real scalar, violating the requirement that the integral over the entire domain must equal one:

$$\int p_{\phi}(\mathbf{x}) d\mathbf{x} = 1$$

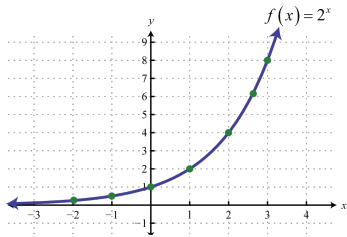
💡 Practical Approach

A network can naturally produce a real scalar $E_{\phi}(\mathbf{x}) \in \mathbb{R}$ for input $\mathbf{x}$.

Step 1: Ensuring Non-Negativity

Apply positive function to raw network output:

$$\tilde{p}_{\phi}(\mathbf{x}) = \exp(E_{\phi}(\mathbf{x}))$$



Step 2: Enforcing Normalization

Divide by integral to create valid density:

$$\begin{aligned} p_{\phi}(\mathbf{x}) &= \frac{\tilde{p}_{\phi}(\mathbf{x})}{\int \tilde{p}_{\phi}(\mathbf{x}') d\mathbf{x}'} \\ &= \frac{\exp(E_{\phi}(\mathbf{x}))}{\int \exp(E_{\phi}(\mathbf{x}')) d\mathbf{x}'} \end{aligned}$$

Normalizing constant (or Partition function)

$$Z(\phi) := \int \exp(E_{\phi}(\mathbf{x}')) d\mathbf{x}'$$

Challenges in Distribution Modeling

Normalizing constant (or Partition function)

$$Z(\phi) := \int \exp(E_{\phi}(\mathbf{x}')) d\mathbf{x}'$$

$$p_{\phi}(\mathbf{x}) = \frac{\tilde{p}_{\phi}(\mathbf{x})}{\int \tilde{p}_{\phi}(\mathbf{x}') d\mathbf{x}'} = \frac{\exp(E_{\phi}(\mathbf{x}))}{\int \exp(E_{\phi}(\mathbf{x}')) d\mathbf{x}'}$$



The Computational Challenge

- For most high-dimensional problems, the integral required to compute the normalizing constant $Z(\phi)$ is **intractable**.
- This intractability is a central problem that **motivates** the development of many different families of deep generative models.

Prominent Deep Generative Models (DGMs)

Energy-Based Models (EBMs)



Central Challenge in generative modeling

How to learn expressive probabilistic models that can capture the rich and complex structure of high-dimensional data.

Energy-Based Models (EBMs) define a probability distribution through an energy function $E_\phi(\mathbf{x})$ that assigns lower energy to more probable data points.

Probability Definition $p_\phi(\mathbf{x}) := \frac{1}{Z(\phi)} \exp(-E_\phi(\mathbf{x}))$



Energy Function

$$E_\phi(\mathbf{x})$$



Partition Function

$$Z(\phi) = \int \exp(-E_\phi(\mathbf{x})) d\mathbf{x}$$

Key Characteristics

- Exponential function ensures non-negativity
- Partition function ensures normalization

Partition Function Challenge

$$Z(\phi) = \int \exp(-E_\phi(\mathbf{x})) d\mathbf{x}$$

- Normalizing constant required for valid probability
- Computational intractability for high-dimensional problems

Autoregressive Models (ARs)

Autoregressive models (ARs) factorize the joint data distribution into a product of conditional probabilities.

✓ **Mathematical Formulation** $p_{\text{data}}(\mathbf{x}) = \prod_{i=1}^D p_{\phi}(x_i | \mathbf{x}_{<i})$ where $\mathbf{x} = (x_1, \dots, x_D)$ and $\mathbf{x}_{<i} = (x_1, \dots, x_{i-1})$.

⚙️ How ARs Work

- Each $p_{\phi}(x_i | \mathbf{x}_{<i})$ parameterized by neural network, such as a Transformer
- Global normalization is trivial
 - ✓ For discrete vars: often use softmax normalization
 $p_{\phi}(x_i | \mathbf{x}_{<i}) = \text{softmax}(\text{logits}_i) \rightarrow \prod_{i=1}^D p_{\phi}(x_i | \mathbf{x}_{<i}) = 1$
 - ✓ For continuous vars: use parameterized Gaussian
 $p_{\phi}(x_i | \mathbf{x}_{<i}) = \mathcal{N}(\mu_{\phi}(\mathbf{x}_{<i}), \sigma_{\phi}(\mathbf{x}_{<i})) \rightarrow \int p_{\phi}(x_i | \mathbf{x}_{<i}) = 1$

✓ Key Properties

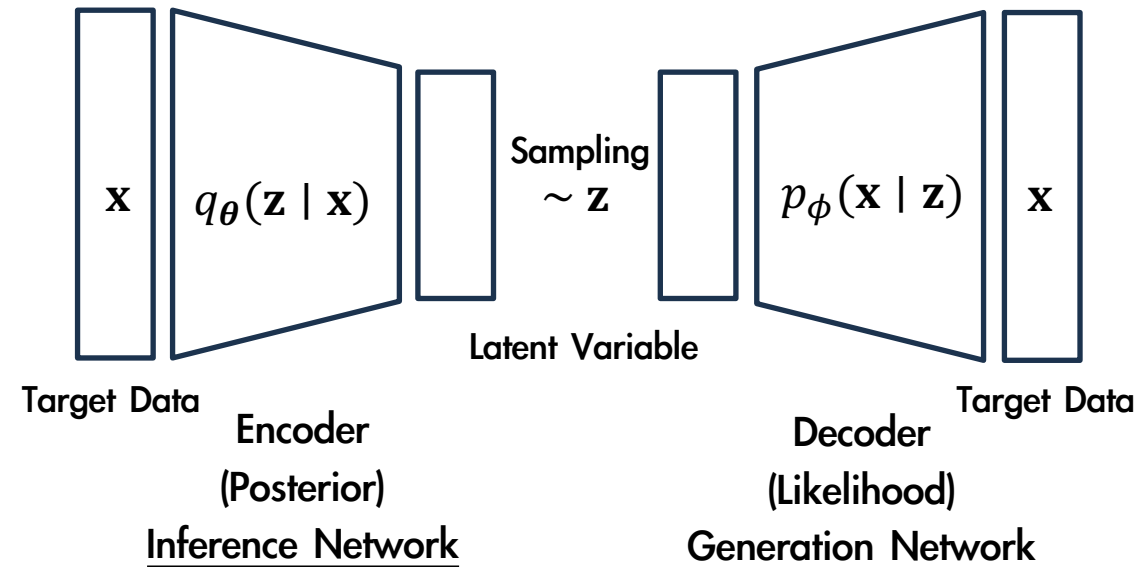
- Strong density estimation capabilities
- Exact likelihood computation
- Sequential generation limits speed
- Fixed ordering may restrict flexibility

Variational Autoencoders (VAEs)

Variational Autoencoders (VAEs) extend classical autoencoders by introducing latent variables z that capture hidden structure in the data x , providing a probabilistic framework for generation

⚙️ How VAEs Work

- No direct mapping between x and z ,
- It learns both an encoder and a decoder
 - 1) Encoder $q_{\theta}(z | x)$ approximates the unknown distribution of latent variables z given the data x
 - 2) Decoder $p_{\phi}(x | z)$ reconstructs data x from these latent variables z
- widely used in likelihood-based generative modeling



✔️ How to train VAEs? → Maximizing Evidence Lower Bound (ELBO)

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi; x) = \underbrace{\mathbb{E}_{z \sim q_{\theta}(z|x)} [\log p_{\phi}(x | z)]}_{\text{accurate reconstruction of the data}} - \underbrace{\mathcal{D}_{\text{KL}}(q_{\theta}(z | x) || p_{\text{prior}}(z))}_{\text{regularizing the latent variables by keeping them close to a simple prior distribution } p_{\text{prior}}(z) \text{ (often Gaussian)}}$$

accurate reconstruction of the data

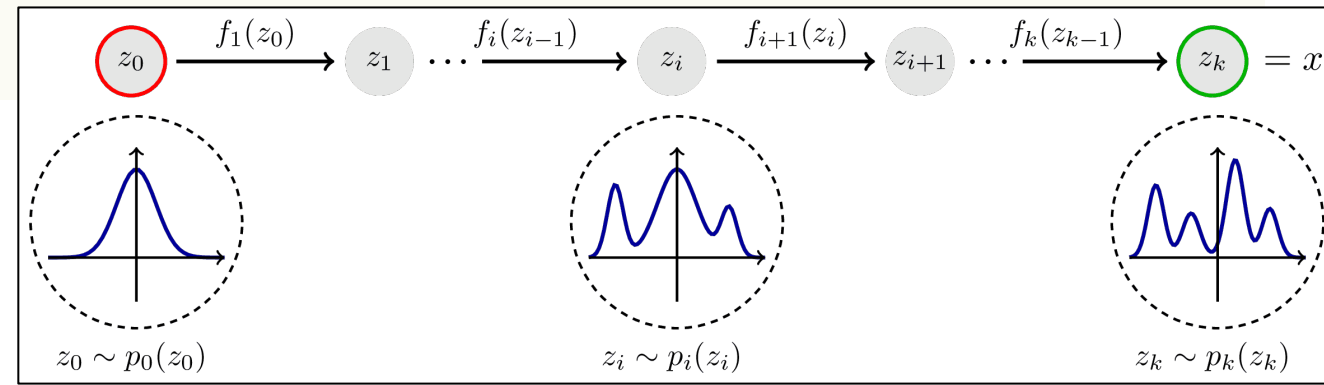
regularizing the latent variables by keeping them close to a simple prior distribution $p_{\text{prior}}(z)$ (often Gaussian)

Normalizing Flows (NFs)

Normalizing Flows (NFs) model a complex data distribution by learning a **invertible transformation** $\mathbf{f}_\phi$ from a simple latent $\mathbf{z}$ to a data $\mathbf{x}$.

- Classic flow-based models, such as **Normalizing Flows (NFs)**
- **Neural Ordinary Differential Equations (NODEs)**

Rezende and Mohamed, 2015; Chen et al., 2018



✓ Key Features

- Explicit modeling of normalized densities
 - **change-of-variable formula for densities**

$$\log p_\phi(\mathbf{x}) = \log p(\mathbf{z}) + \log \left| \det \frac{\partial \mathbf{f}_\phi^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right|$$

- Invertible transformations with tractable Jacobian determinants
- Exact and tractable likelihood computation
- Analytical absorption of normalization constant

⚠ Practical Challenges

- NFs impose restrictive architectural constraints to ensure bijectivity
- NODEs have computational overhead for solving ODEs
- Scaling challenges with high-dimensional data
- Training inefficiencies compared to other approaches

Generative Adversarial Networks (GANs)

GANs consist of two neural networks, a **generator** and a **discriminator** that compete against each other

- They do not define an explicit density function and bypass likelihood estimation
- Instead of computing a normalization constant, GANs focus on generating samples that closely mimic the data distribution

Generator $G_\phi(\mathbf{z})$

- Create realistic samples $G_\phi(\mathbf{z})$ from random noise $\mathbf{z} \sim p_{prior}$
- Goal: Generate samples that mimic the data distribution

Discriminator $D_\xi(\mathbf{z})$

- distinguish between real samples $\mathbf{x}$ and generated samples $G_\phi(\mathbf{z})$
- Outputs probability that a sample is real (not generated)

© GAN Objective Function

$$\min_{G_\phi} \max_{D_\xi} \underbrace{\mathbb{E}_{\mathbf{x} \sim p_{data}(\mathbf{x})} [\log D_\xi(\mathbf{x})]}_{\text{real}} + \underbrace{\mathbb{E}_{\mathbf{z} \sim p_{prior}(\mathbf{z})} [\log (1 - D_\xi(G_\phi(\mathbf{z})))]}_{\text{fake}}$$

Generative Adversarial Networks (GANs)

✓ Analysis of GANs

- an optimal discriminator for a fixed generator $G_\phi \longrightarrow D_\xi(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{G_\phi}(\mathbf{x})}$
- Then, the objective function is reduced to

$$\min_{G_\phi} 2\mathcal{D}_{\text{JS}}(p_{\text{data}} \parallel p_{G_\phi}) - \log 4.$$

where $\mathcal{D}_{\text{JS}}$ denotes the Jensen–Shannon divergence

즉, GAN은 분류 게임을 하고 있지만, 실제로는 JS divergence를 줄이도록 Generator를 학습시키고 있음

- More broadly, extensions such as f -GANs generalize this view by demonstrating that GAN training can minimize a family of f -divergences, placing GANs within the same divergence-minimization framework as other generative models.

f -GAN은 GAN 구조를 확장해서 JS뿐 아니라 임의의 f -divergence를 최소화하여 학습할 수 있음

- GAN의 단점
 - min-max training process $\rightarrow$ notoriously unstable, often requiring carefully designed architectures & engineering techniques $\rightarrow$ 게다가, Mode Seeking 현상 발생

Taxonomy of Modelings

Explicit vs. Implicit Models

A fundamental distinction in deep generative models (DGMs) is whether 1) they directly parameterize the probability distribution or 2) only specify it through sampling.

Explicit Models

- Directly parameterize the probability distribution $p_{\phi}(\mathbf{x})$
- Can evaluate the density $p_{\phi}(\mathbf{x})$ directly
- Examples: ARs, NFs, VAEs, DMs

✓ Tractable likelihood

Implicit Models

- Specify distribution through a sampling procedure $\mathbf{x} = \mathbf{G}_{\phi}(\mathbf{z})$
- May not have a closed-form density $p_{\phi}(\mathbf{x})$
- Examples: GANs

✏ Sampling-based generation

[Comparison Table]

Model Type	Likelihood	Objective	Examples
Explicit	Tractable	MLE	NFs, ARs
Explicit	Bound/Approx.	ELBO	VAEs, DMs
Implicit	Not Directly Modeled	Adversarial	GANs

Diffusion Models: Three Perspectives

Diffusion Model은 완전히 새로운 모델이 아니라, 기존 생성모델들의 아이디어를 한데 엮은 프레임워크



Variational Autoencoders

Connection: Diffusion models use variational training objectives

- Hierarchical latent variable structure
- Encoder-decoder architecture
- Evidence Lower Bound (ELBO) training
- Denoising Diffusion Probabilistic Models (DDPMs)



Energy-Based Models

Connection: Diffusion models use score-matching approaches

- Learn gradients of log-density
- Energy function parameterization
- Score-based generative models
- Noise Conditional Score Networks (NCSN)



Normalizing Flows

Connection: Diffusion models use continuous-time transformations

- Invertible transformations
- Tractable Jacobian determinants
- Explicit density modeling
- Flow Matching (FM)

Diffusion models unify these three perspectives into a coherent framework for generative modeling



Unified Diffusion Framework

Common Principles

- Stochastic differential equations
- Multi-step generation process

Key Innovations

- Bridge between discrete and continuous models
- Unified treatment of generation

Practical Impact

- Improved sample quality
- Efficient training and inference

DGMs & Diffusion Models (DMs)

